

lines: A list of strings to append as individual lines.

Returns:

The updated working memory string.

deletes_from_memory

Deletes lines from the given section using robust, plain-text matching.

Args:

section_title: The section to edit (e.g., "Goals and Plans", "Facts and Knowledge", "Active Notes").

lines: A list of target lines to remove.

Returns:

The updated working memory string.

patch_memory

Apply a context-based patch to the working memory string (no line numbers).

Each patch contains one or more hunks, *anchored to a memory section* using a header of the form: `@@ section: <Title>`.

Within a section, specify deleted lines with '-' and added lines with '+', optionally including 1-3 unchanged context lines before/after for disambiguation.

- If a section or target is ambiguous/missing, the tool FAILS with a diagnostic.
- The patch should be IDEMPOTENT: re-applying it should not corrupt memory.
- Returns the updated memory plus metadata (applied hunks, changed lines, etc.).

Required args:

- patch (string): The patch text with the required headers:

```
```
*** Begin Patch
*** Update Memory
@@ section: Goals and Plans
- Old line
+ New line
*** End Patch
```
```

- explanation (string): one-sentence high-level intent for the change.

Optional safety args:

- expected_hunks (int): how many hunks you intend to apply.
- expected_changes (int): total +/- line operations expected.
- options (dict):
 - * strict_context (bool, default False): require clear context or exact old block.
 - * normalize_whitespace (bool, default True): collapse repeated spaces when matching.
 - * case_sensitive (bool, default True): literal, case sensitive matching when True.

Integration arg (injected by agent):

- current_memory (string): the current working-memory string to patch.

Returns:

```

{
  "new_memory": <updated string>,
  "meta": {
    "applied_hunks": <int>,
    "changed_lines": <int>,
    "sections_touched": [<str>...],
    "warnings": [<str>...]
  }
}

```

Few-shot examples

Example A: simple replacement within a section

```

*** Begin Patch
*** Update Memory
@@ section: Goals and Plans
- Planned steps or strategies for achieving them.
+ Planned steps: finish MVP, write docs, ship v1 by Sept 15.
*** End Patch

```

Example B: multi-line update with light context

```

*** Begin Patch
*** Update Memory
@@ section: Active Knowledge
[Investigation notes]
- Hypothesis: caching is the bottleneck
- Evidence: slow Redis reads
+ Hypothesis (confirmed): caching is the bottleneck
+ Evidence: slow Redis reads; profiler shows 45% time in cache layer
*** End Patch

```

Example C: pure insertion (anchor with a nearby context line)

```

*** Begin Patch
*** Update Memory
@@ section: Goals and Plans
[Milestones]
+ - Milestone: pass integration tests in CI
*** End Patch

```

replace_in_memory

Replace an exact text span in the working memory. Prefer scoping by `section\_title` and include light `pre\_context` and/or `post\_context` to uniquely anchor the target. By default the tool replaces exactly ONE occurrence (set `expected\_replacements` when intentionally replacing multiple). Fails if the target is ambiguous or not found.

Returns the updated memory and metadata.

Arguments:

- old_string (str, required): exact literal text to replace.
- new_string (str, required): exact literal replacement.
- explanation (str, required): one-sentence intent for telemetry.
- section_title (str, optional): limit search to a single section; matches titles

```

    like '## n. <Title>' ignoring the numeric prefix (case-insensitive on the title
    text).
- expected_replacements (int, optional; default=1): number of replacements
intended.
- pre_context (str, optional): unchanged text that must appear immediately before
each target.
- post_context (str, optional): unchanged text that must appear immediately after
each target.
- options (dict, optional): same as patch_memory:
    * strict_context (bool, default True): require unambiguous anchors; otherwise
    fail.
    * normalize_whitespace (bool, default False): collapse repeated spaces when
    matching (matching only).
    * case_sensitive (bool, default True): literal case-sensitive matching.

- current_memory (str, injected by agent): the working-memory string to edit.

```

Returns:

```

{
  "new_memory": <updated string>,
  "meta": {
    "applied_hunks": 1,
    "changed_lines": <int>,
    "sections_touched": [<str>...],
    "warnings": [<str>...]
  }
}

```

Examples:

```

# Replace a single bullet inside "Goals and Plans"
{
  "section_title": "Goals and Plans",
  "old_string": "- Current overarching goal.",
  "new_string": "- Current overarching goal: ship v1 by Sept 15.",
  "expected_replacements": 1,
  "explanation": "Clarify primary goal"
}

# Multi-occurrence replacement in a section, with context
{
  "section_title": "Active Knowledge (Reasoning Outputs & Ephemeral Notes)",
  "pre_context": "[Investigation notes]",
  "old_string": "Hypothesis: caching is the bottleneck",
  "new_string": "Hypothesis (confirmed): caching is the bottleneck",
  "expected_replacements": 2,
  "explanation": "Mark hypothesis as confirmed"
}

```